DevOps

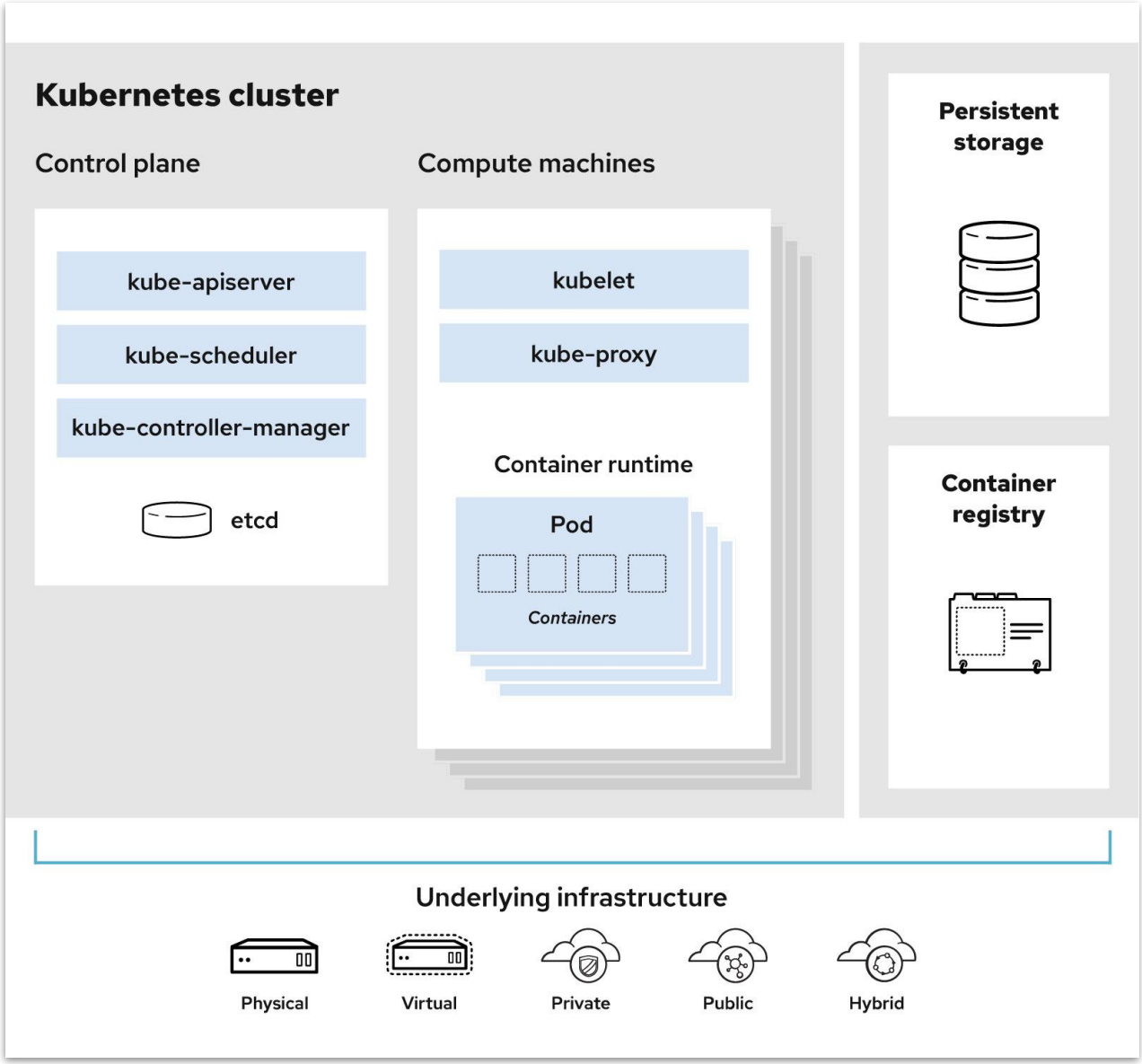
Kubernetes Overview

What is Kubernetes?

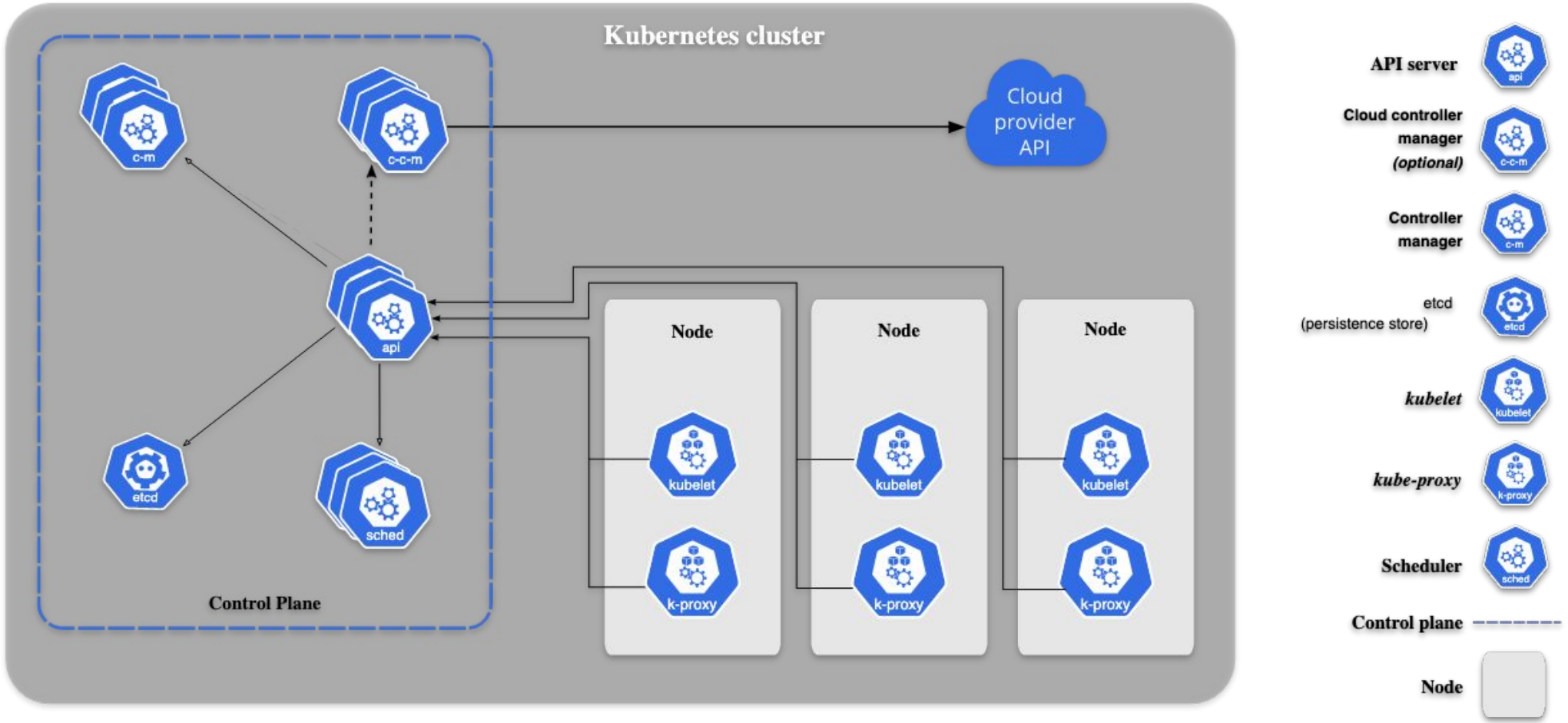
- Kubernetes, also known as K8s, is an open-source Container Orchestration Tool for automating deployment, scaling, and management of containerized applications.
- Container orchestration automates most of the tasks required to run containerized workloads and services, including the operations that are essential to the Kubernetes container lifecycle: provisioning, deployment, scaling, networking and load balancing.
- What are Kubernetes clusters?

  You can cluster together groups of hosts running Linux containers, and Kubernetes helps you easily and efficiently manage those clusters.

Kubernetes Architecture



Kubernetes Architecture



Kubernetes Architecture

- Control plane: The collection of processes that control Kubernetes nodes. This is where all task assignments originate.
- Nodes: These machines perform the requested tasks assigned by the **control plane**.
- Pod: A group of one or more containers deployed to a single node. All containers in a pod share an IP address, IPC, hostname, and other resources. Pods abstract network and storage from the underlying container. This lets you move containers around the cluster more easily.
- Replication controller: This controls how many identical copies of a pod should be running somewhere on the cluster.
- Service: This decouples work definitions from the pods. Kubernetes service proxies automatically get service requests to the right pod—no matter where it moves in the cluster or even if it's been replaced.

Kubernetes Architecture

- Kubectl: The command line configuration tool for Kubernetes.
- Cloud-controller-manager: A Kubernetes **control plane** component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster.
- Kube-apiserver is a component of the Kubernetes **control plane** that exposes the Kubernetes API.
- Kube-scheduler: Control plane component that watches for newly created **Pods** with no assigned **node**, and selects a node for them to run on.

Kubernetes Architecture

- Etcd is A key-value store where all data relating to the Kubernetes cluster is stored.
- Kube-controller-manager: All controller functions of the control plane are compiled into a single binary: kube-controller-manager.
- Kubelet: An agent that makes sure that the necessary containers are running in a Kubernetes Pod.
- Kube-proxy: A network proxy that runs on each node in a cluster to maintain network rules and allow communication.
- Container runtime: The software responsible for running containers. Kubernetes supports any runtime that adheres to the Kubernetes CRI (Container Runtime Interface).

Kubernetes Features

- Auto-scaling: Automatically scale containerized applications and their resources up or down based on usage.
- Lifecycle management: Automate deployments and updates with the ability to:
  - Rollback to previous versions
  - Pause and continue a deployment
- Declarative model: Declare the desired state, and K8s works in the background to maintain that state and recover from any failures
- Resilience and self-healing: Auto placement, auto restart, auto replication and auto scaling provide application self-healing
- Persistent storage: Ability to mount and add storage dynamically
- Load balancing: Kubernetes supports a variety of internal and external load balancing options to address diverse needs

Kubernetes Features

- DevSecOps support: DevSecOps is an advanced approach to security that simplifies and automates container operations across clouds, integrates security throughout the container lifecycle, and enables teams to deliver secure, high-quality software more quickly. Combining DevSecOps practices and Kubernetes improves developer productivity.
- Such as
    - Secret Management
    - Role-based Access Control (RBAC)
    - Network Policies ....

Kubernetes Automatic Cluster Installation – Kubespray

- Resources:
  - https://www.youtube.com/watch?v=omJ4Vzudvog
  - https://schoolofdevops.github.io/ultimate-kubernetes-bootcamp/cluster_setup_kubespray/
  - https://github.com/kubernetes/dashboard/blob/master/docs/user/access-control/creating-sample-user.md
  - https://argo-cd.readthedocs.io/en/stable/getting_started/
- Create new server — master

```
Use user only with ssh-keys (do not use “PasswordAuthentication Yes”)
$ sudo apt update
$ sudo vim /etc/sysctl.conf
  -   Add "net.ipv4.ip_forward=1" to the last line
```

Kubernetes Automatic Cluster Installation – Kuberspray

- Create nodes from image: Totally 3 or 5 nodes
- set static ip
- On master server

```
$ generate ssh-key # Do not set password
 -  Copy ssh key to all servers + ownself
$ sudo dpkg-reconfigure locales
$ git clone https://github.com/kubernetes-incubator/kubespray.git
$ cd kubespray
$ sudo apt update
$ sudo apt install python3-pip -y
$ sudo pip3 install -r requirements.txt
$ ansible --version
$ sudo vim /home/dara/kubespray/ansible.cfg
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
[ssh_connection]
pipelining=True
ssh_args = -o ControlMaster=auto -o ControlPersist=30m -o ConnectionAttempts=100 -o
UserKnownHostsFile=/dev/null
#control_path = ~/.ssh/ansible-%%r@%%h:%%p
[defaults]
host_key_checking=False
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp
stdout_callback = skippy
library = ./library
callback_whitelist = profile_tasks
roles_path =
roles:$VIRTUAL_ENV/usr/local/share/kubespray/roles:$VIRTUAL_ENV/usr/local/share/ansible/roles
deprecation_warnings=False
remote_user=dara
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
$ cd kubespray
$ cp -rfp inventory/sample inventory/prod
$ CONFIG_FILE=inventory/prod/hosts.ini python3 contrib/inventory_builder/inventory.py
10.10.1.101 10.10.1.102 10.10.1.103 10.10.1.104 10.10.1.105
$ cd kubespray/inventory/prod
$ vim hosts.ini
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
#Use Private IP
[all]
node1    ansible_host=10.10.1.101 ip=10.10.1.101
node2    ansible_host=10.10.1.102 ip=10.10.1.102
node3    ansible_host=10.10.1.103 ip=10.10.1.103
node4    ansible_host=10.10.1.104 ip=10.10.1.104
node5    ansible_host=10.10.1.105 ip=10.10.1.105

[kube-master]
node1
node2
node3

[kube-node]
node1
node2
node3
node4
node5
```

```
[etcd]
node1
node2
node3


[k8s-cluster:children]
kube-node
kube-master

[calico-rr]

[vault]
node1
node2
node3
node4
node5
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
$ cd kubespray/inventroy/prod/group_vars/k8s_cluster
$ sudo vim k8s-cluster.yml
```

```
# Add these to the last line
kubelet_max_pods: 100
cluster_name: prod # find the same name and comment it
helm_enabled: true
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
$ sudo vim addons.yml
```

```
+ dashborad_enabled: true
+ helm_enabled: true
+ metrics_server_enabled: true
+ ingress_nginx_enabled: true
+ cert_manager_enabled: true
+ argocd_enabled: true
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
$ cd kubespray
$ ansible-playbook -b -v -i inventory/prod/hosts.ini cluster.yml
$ mkdir -p $HOME/.kube
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
$ kubectl get all -A
$ kubectl edit service/kubernetes-dashboard -n kube-system
    + Change => type: NodePort
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
$ vim kubernetes-dashbord.yml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
 name: admin-user
 namespace: kubernetes-dashboard
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: admin-user
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: cluster-admin
subjects:
- kind: ServiceAccount
  name: admin-user
     namespace: kubernetes-dashboard
```

Kubernetes Automatic Cluster Installation – Kuberspray

```
$ kubectl create ns kubernetes-dashboard
$ kubectl apply -f kubernetes-dashbord.yml
$ kubectl -n kubernetes-dashboard create token admin-user --duration=24h
```

- Copy token and login in Kubernetes Dashboard

- Change configuration for ingress nginx

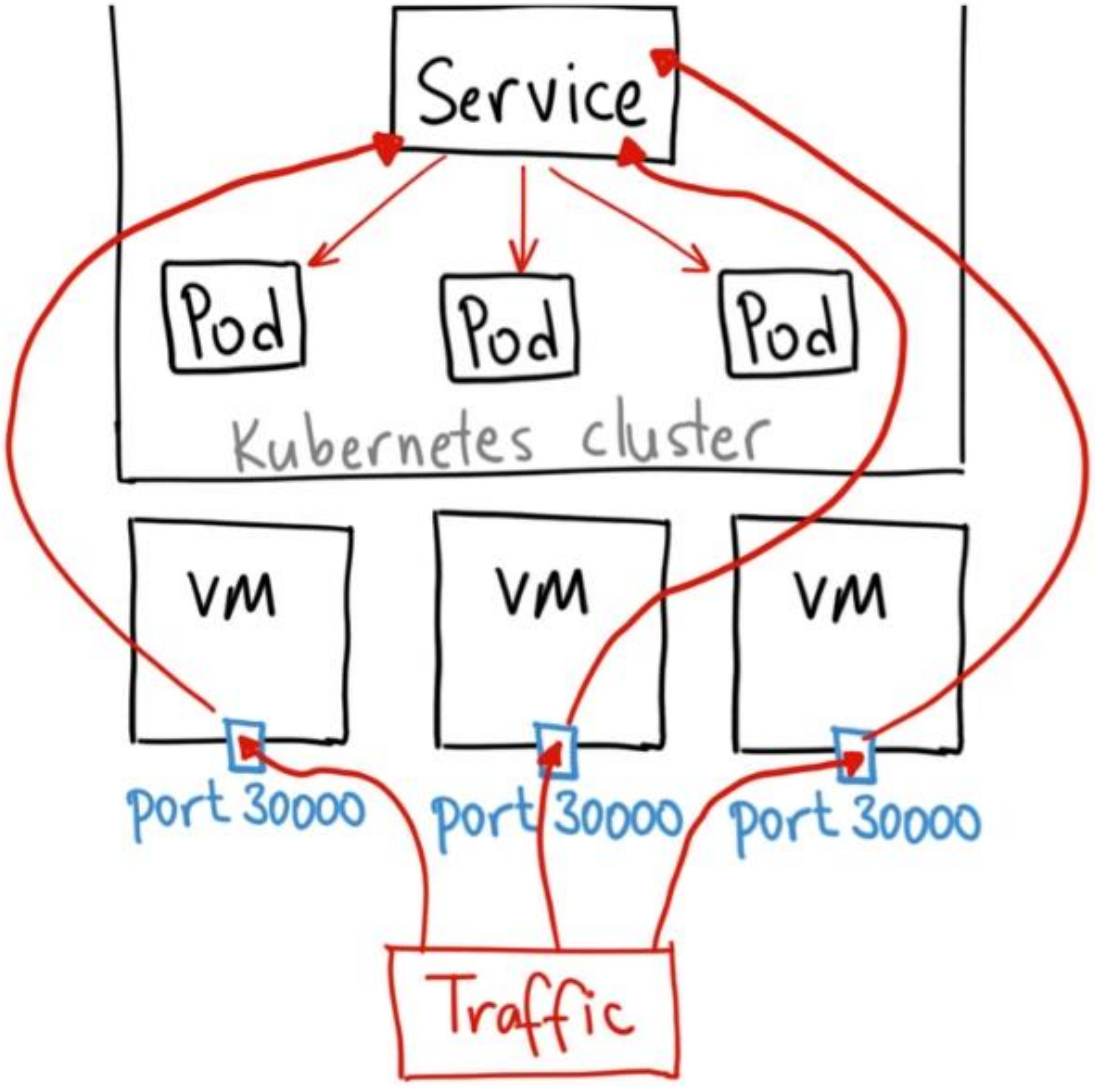
  - Drop Down: Change to "ingress-nginx"
  - Go to Tab: Daemon Sets => nginx-ingress-controller => edit
  - Go to spec: => containers: => args:
  - Add theses line with all workers private IP

```
- '--watch-ingress-without-class=true'
- '--publish-status-address=10.10.1.101,10.10.1.102,10.10.1.103'
```

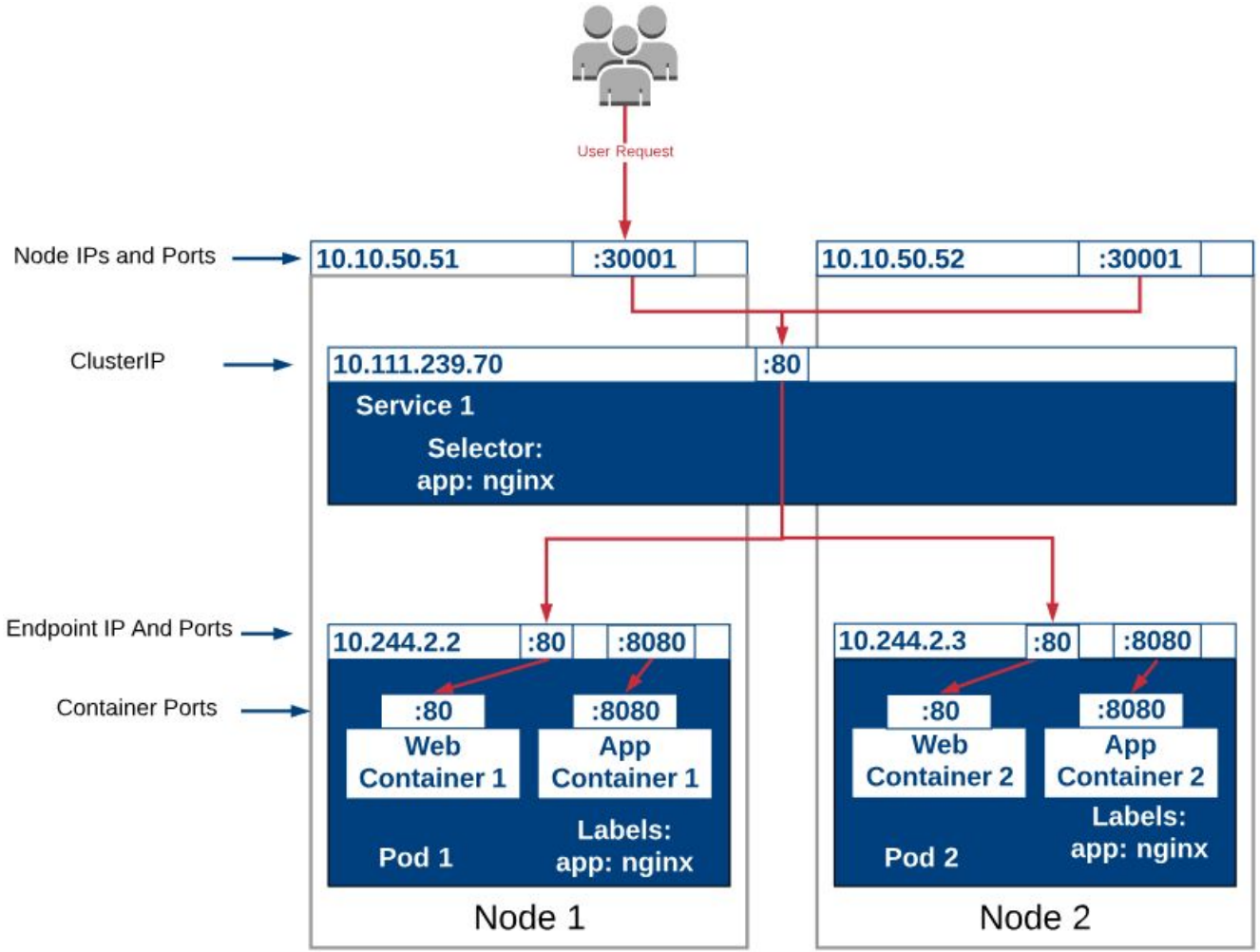
- Configuration for ArgoCD

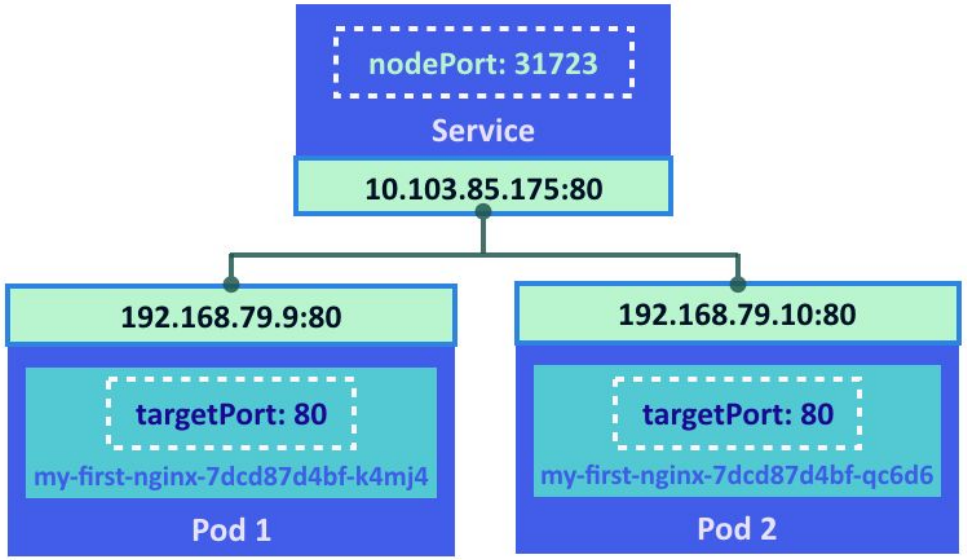
```
$ kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
```

Kubernetes Ports (30000 ~ 32767)



Kubernetes Ports (30000 ~ 32767)





Kubernetes Ports (30000 ~ 32767)

```yaml
kind: Service
apiVersion: v1

metadata:
  name: hostname-service

spec:
  type: NodePort
  selector:
    app: echo-hostname
  ports:
    - nodePort: 30163
      port: 8080
      targetPort: 80
```

Make the service available to network requests from external clients

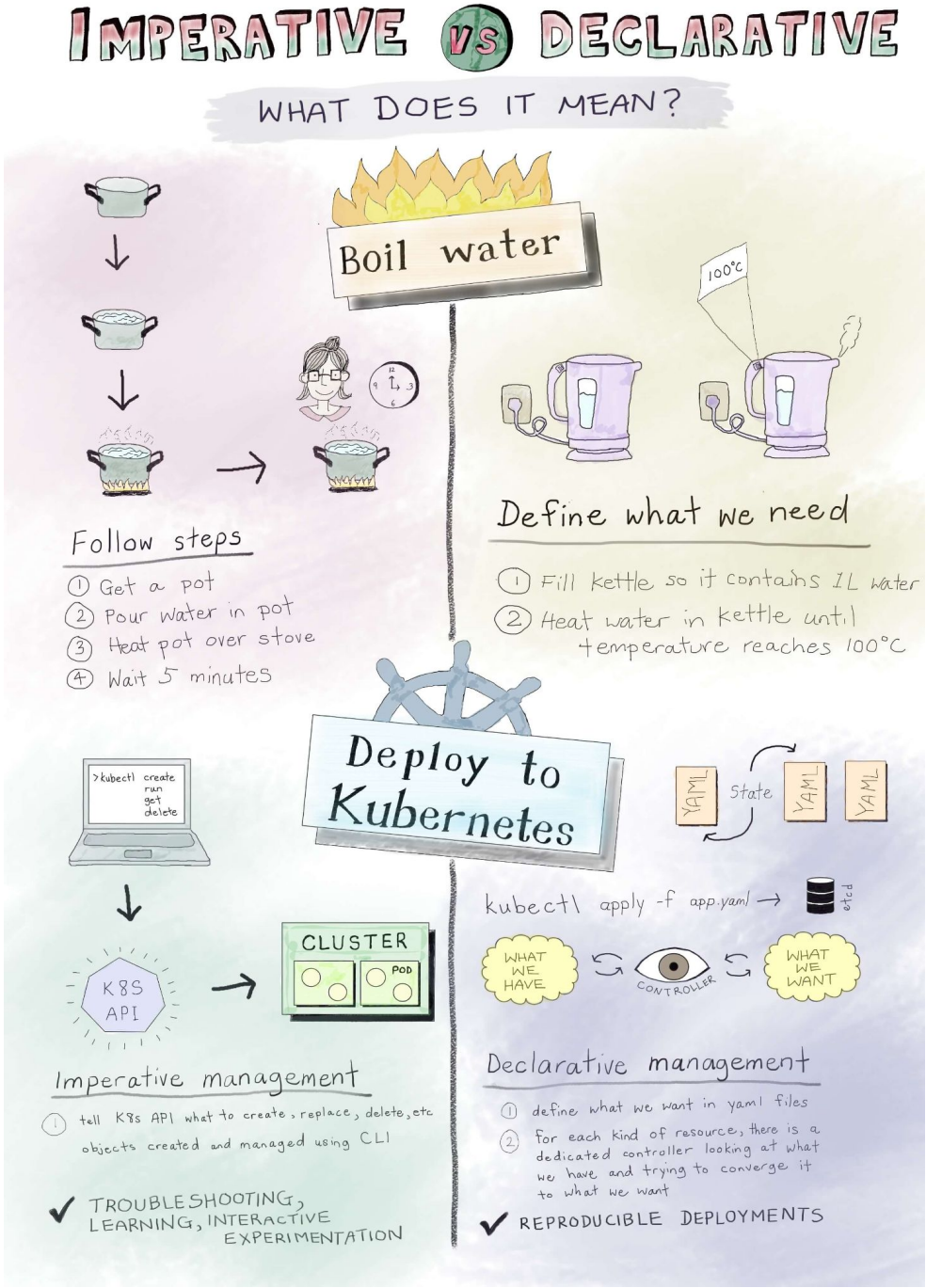
Forward requests to pods with label of this value

`nodePort`
access service via this external port number

`port`
port number exposed internally in cluster

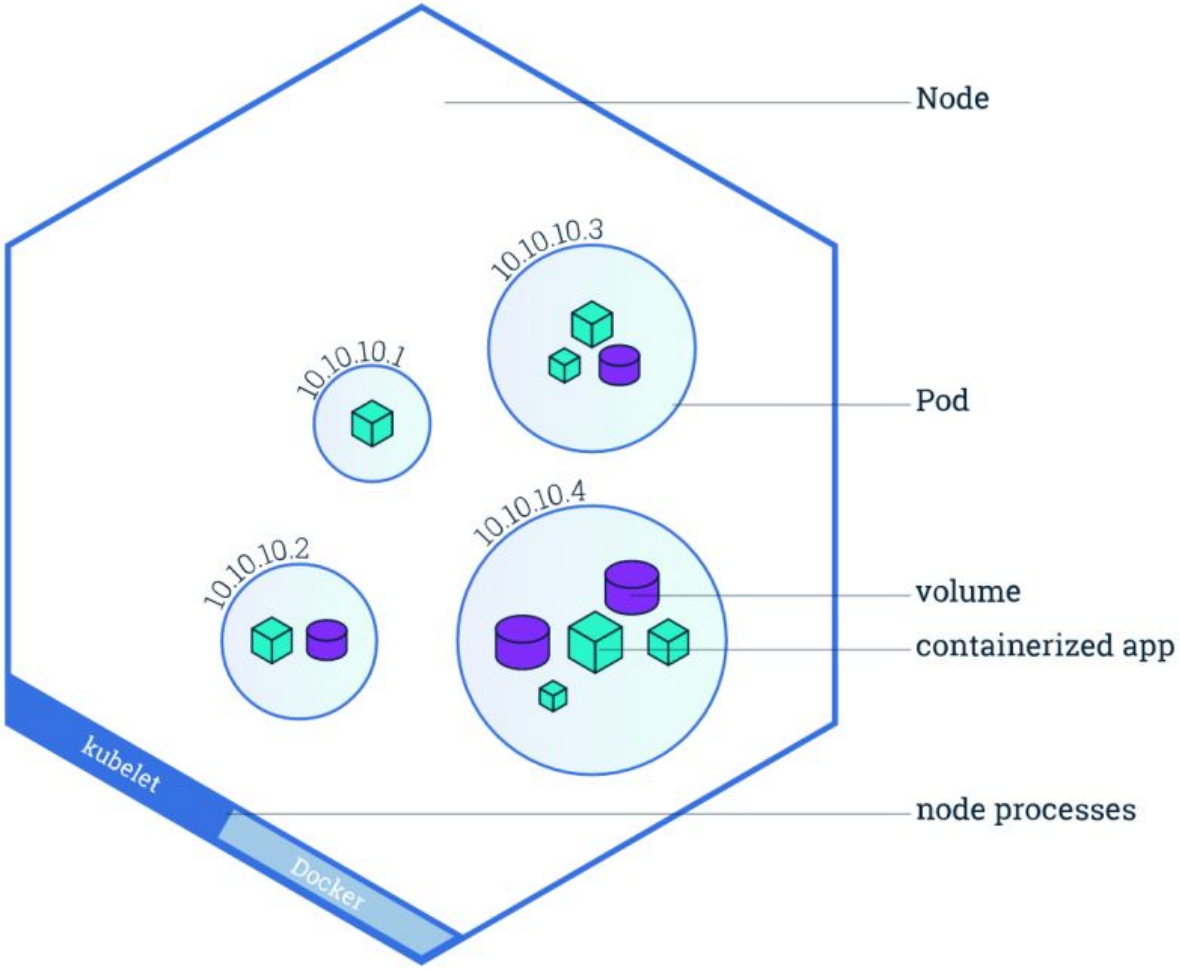
`targetPort`
port that containers are listening on

Kubernetes Objects



Kubernetes Objects

- Pod: are the smallest deployable units of computing that you can create and manage in Kubernetes.



Kubernetes Objects

- Imperative

  Docs: https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#run

  ```
  $ kubectl run my-nginx --image=nginx:1.22.1 --restart=Never
  ```

- Declarative

```yaml
# my-pod.yaml
apiVersion: v1
kind: Pod
metadata:
 name: nginx-pod
 labels:
   app: nginx-lbl
spec:
 containers:
   - name: nginx-cont
     image: nginx:1.22.1
     ports:
       - containerPort: 80
 restartPolicy: Never
```

command	object does not exist	object already exists
create	create new object	ERROR
apply	create new object (needs complete spec)	configure object (accepts partial spec)
replace	ERROR	delete object create new object
patch	ERROR	configure object (accepts partial spec)

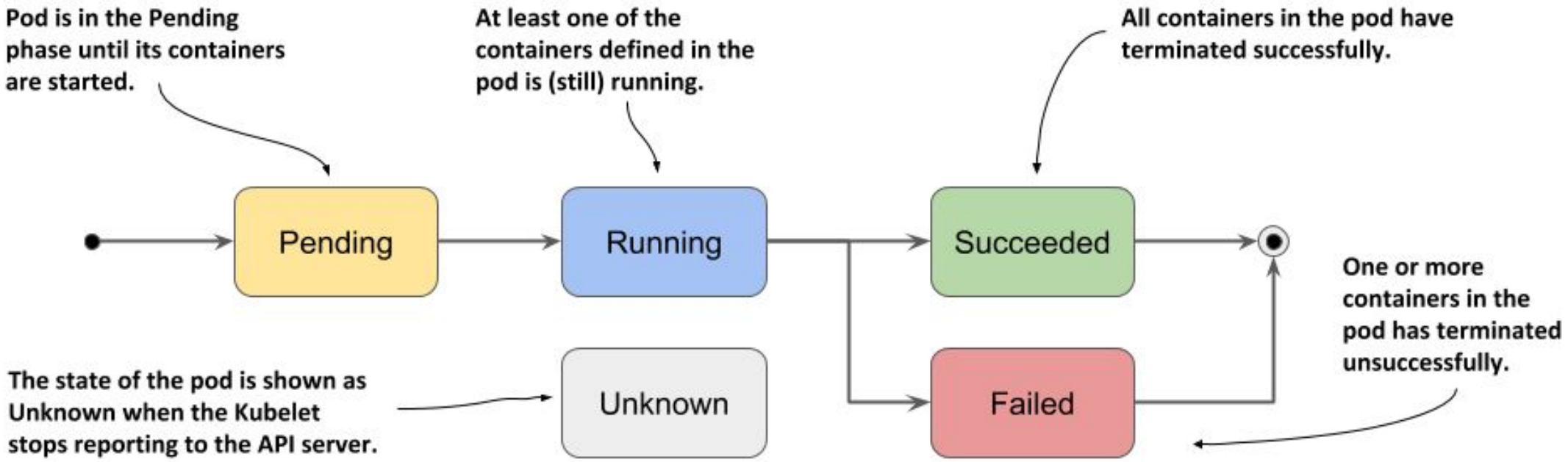
Kubernetes Objects

- Run: `$ kubectl apply -f my-pod.yaml`
- Some Commands about Pod

```
kubectl api-resources # Show all Kubernetes Objects API
kubectl get all -A # -A: --all-namespace
kubectl get pod
kubectl get pod/nginx-pod -o yaml # -o yaml or -o json > pod.yaml
kubectl describe pod/nginx-pod # Detail about Pod
kubectl logs pod/nginx-pod
kubectl exec -it nginx-pod -- bash
kubectl delete pod nginx-pod # kubectl delete pod/nginx-pod
```

Kubernetes Objects

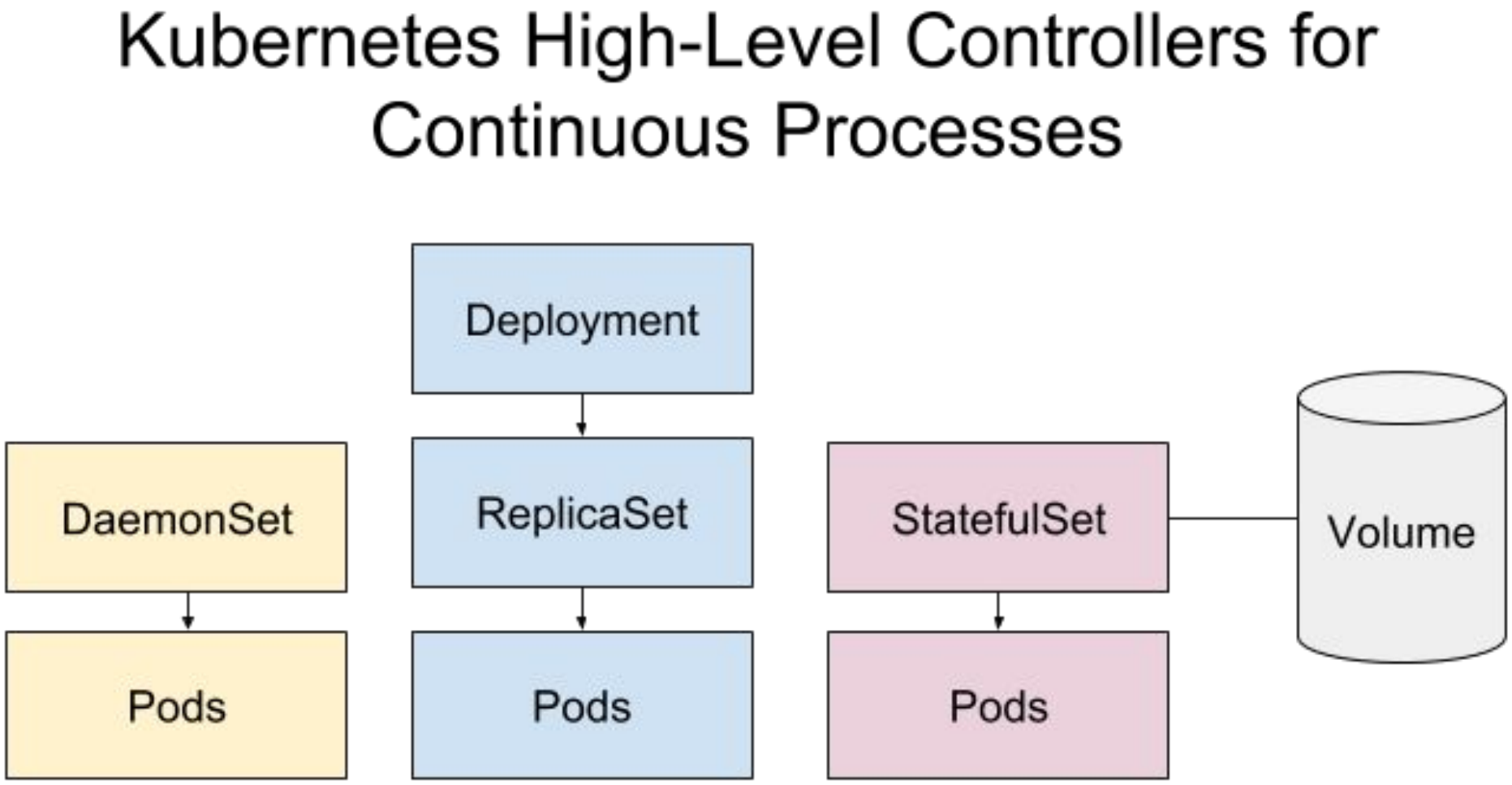
- Pod Lifecycle



Kubernetes Objects

Value	Description
`Pending`	The Pod has been accepted by the Kubernetes cluster, but one or more of the containers has not been set up and made ready to run. This includes time a Pod spends waiting to be scheduled as well as the time spent downloading container images over the network.
`Running`	The Pod has been bound to a node, and all of the containers have been created. At least one container is still running, or is in the process of starting or restarting.
`Succeeded`	All containers in the Pod have terminated in success, and will not be restarted.
`Failed`	All containers in the Pod have terminated, and at least one container has terminated in failure. That is, the container either exited with non-zero status or was terminated by the system.
`Unknown`	For some reason the state of the Pod could not be obtained. This phase typically occurs due to an error in communicating with the node where the Pod should be running.

Kubernetes Objects



Kubernetes Objects

- ReplicaSet

  A ReplicaSet's purpose is to maintain a stable set of replica Pods running at any given time. As such, it is often used to guarantee the availability of a specified number of identical Pods.

```yaml
# my-replicaset.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: frontend
 labels:
   app: nginx-ui
   tier: frontend
spec:
 replicas: 3
 selector:
   matchLabels:
     tier: frontend
```

```yaml
 template:
   metadata:
     labels:
       tier: frontend
   spec:
     containers:
       - name: nginx-ui-cont
         image: nginx:1.22.1
         ports:
           - containerPort: 80
```

```
$ kubectl apply -f my-replicaset.yaml
```

Kubernetes Objects

- Deployment

  A Kubernetes Deployment tells Kubernetes how to create or modify instances of the pods that hold a containerized application. Deployments can help to efficiently scale the number of replica pods, enable the rollout of updated code in a controlled manner, or roll back to an earlier deployment version if necessary.

Kubernetes Objects

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: frontend-web
 labels:
   app: nginx-ui
   tier: frontend-web
spec:
 replicas: 2
 strategy:
  # type: Recreate or RollingUpdate(default)
   rollingUpdate:
     # Create and Remove at the same time
     maxSurge: 1 # create default 25%
     maxUnavailable: 1 # remove default 25%
 minReadySeconds: 5
 # When pod ready and wait 5s
 selector:
   matchLabels:
     tier: frontend
```

```yaml
template:
   metadata:
     labels:
       tier: frontend
   spec:
     containers:
       - name: nginx-web-cont
         image: nginx:1.22.1
         ports:
           - containerPort: 80
```

Kubernetes Objects

- What are Kubernetes Deployment Strategies?
  a. Recreate Deployment (Strategy Type)

     The recreate strategy terminates pods that are currently running and ‘recreates’ them with the new version. This approach is commonly used in a development environment where user activity isn’t an issue. Because the recreate deployment entirely refreshes the pods and the state of the application, you can expect downtime due to the shutdown of the old deployment and the initiation of new deployment instances.

Kubernetes Objects

b. Rolling Update Deployment (Strategy Type)

The rolling update deployment provides an orderly, ramped migration from one version of an application to a newer version. A new ReplicaSet with the new version is launched, and replicas of the old version are terminated systematically as replicas of the new version launch. Eventually, all pods from the old version are replaced by the new version.

The rolling update deployment is beneficial because it provides an organized transition between versions. However, it can take time to complete.

Kubernetes Objects

c. Blue/Green Deployment (Process)

The Blue/Green strategy offers a rapid transition from the old to new version once the new version is tested in production. Here the new ‘green’ version is deployed along with the existing ‘blue’ version. Once there is enough confidence that the ‘green’ version is working as designed, the version label is replaced in the selector field of the Kubernetes Service object that performs load balancing. This action immediately switches traffic to the new version. The Kubernetes blue/green deployment option provides a rapid rollout that avoids versioning issues. However, this strategy doubles the resource utilization since both versions run until cutover.

Kubernetes Objects

d. Canary Deployment (Process)

In a canary deployment, a small group of users is routed to the new version of an application, which runs on a smaller subset of pods. The purpose of this approach is to test functionality in a production environment. Once satisfied that testing is error-free, replicas of the new version are scaled up, and the old version is replaced in an orderly manner. Canary deployments are beneficial when you want to test new functionality on a smaller group of users. Since you can easily roll back canary deployments, this strategy helps gauge how new code will impact the overall system operation without significant risk.

Auto Scaling the Deployment

```
$ kubectl -n kube-system edit deploy metrics-server
```

- Add these lines in YAML File

```
        command:
        - /metrics-server
        - --kubelet-insecure-tls
        - --kubelet-preferred-address-types=InternalIP
```

```
containers:
- args:
  - --logtostderr
  - --cert-dir=/tmp
  - --secure-port=10250
  - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
  - --kubelet-use-node-status-port
  - --kubelet-insecure-tls
  - --metric-resolution=15s
  command:
  - /metrics-server
  - --kubelet-insecure-tls
  - --kubelet-preferred-address-types=InternalIP
  image: registry.k8s.io/metrics-server/metrics-server:v0.6.3
```

Auto Scaling the Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-nginx-deploy
spec:
 replicas: 4
 strategy:
   type: RollingUpdate
   rollingUpdate:
     maxSurge: 1
     maxUnavailable: 1
 selector:
   matchLabels:
     app: my-nginx-app
```

```yaml
template:
  metadata:
    labels:
      app: my-nginx-app
  spec:
    containers:
      - name: my-nginx-cont
        image: nginx:1.22.1
        resources:
          requests:
            cpu: 250m
            memory: "512Mi"
          limits:
            cpu: 500m
            memory: "1Gi"
        ports:
          - containerPort: 80
```

Auto Scaling the Deployment

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: my-nginx-autoscaler
spec:
 scaleTargetRef:
   apiVersion: apps/v1
   kind: Deployment
   name: my-nginx-deploy
 minReplicas: 2
 maxReplicas: 10
 metrics:
 - type: Resource
   resource:
     name: cpu
     target:
       type: Utilization
       averageUtilization: 1
```

Auto Scaling the Deployment

- $ kubectl expose deployment.apps/my-nginx-deploy --name my-nginx-svc --port=80 --type=ClusterIP

- $ kubectl top node
- $ kubectl get all
- $ kubectl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- 10.233.41.255:80; done"
  - Use in another Terminal
- $ kubectl get hpa my-nginx-autoscaler --watch

Thank you

Begin with the theory, Start with the practical